

CryptoCarver

Plan de evolución funcional, técnica y de interfaz

Versión del plan: 1.0

Fecha: 12 de julio de 2026

Horizonte: 12 meses

Documento vivo para priorización y seguimiento

Una hoja de ruta para evolucionar CryptoCarver como laboratorio criptográfico modular, explicable, reproducible y verificable.

Contenido

1. Resumen ejecutivo	2. Principios de producto
3. Estado actual resumido	4. Arquitectura objetivo
5. Roadmap por fases	6. Backlog transversal priorizado
7. Estrategia de pruebas	8. Modelo de versiones recomendado
9. Siguiendo sprint recomendado	10. Decisiones que conviene posponer
11. Indicadores de evolución	12. Definition of Done
13. Referencias técnicas	14. Mantenimiento del roadmap

1. Resumen ejecutivo

CryptoCarver ya cubre un conjunto amplio de operaciones criptográficas: conversiones, cifrado simétrico y asimétrico, firmas, certificados, ASN.1, CMS, JOSE, pagos, EMV, TR-31, XAdES y criptografía post-cuántica. El siguiente salto de calidad no depende solamente de añadir más algoritmos. La prioridad debe ser convertir el conjunto actual en una plataforma coherente, verificable y fácil de ampliar.

El plan propone trabajar sobre cinco líneas simultáneas:

1. **Estabilidad y confianza:** eliminar duplicidades, cerrar operaciones incompletas, mejorar errores y ampliar pruebas con vectores conocidos.
2. **Experiencia de usuario:** hacer que cada pantalla explique claramente la operación, los formatos, la codificación y los resultados.
3. **Profundidad criptográfica:** completar XAdES, PQC, TR-31, EMV, JOSE y gestión de claves con interoperabilidad real.
4. **Automatización:** permitir lotes, recetas reproducibles, CLI y exportación de resultados.
5. **Distribución y mantenimiento:** empaquetado multiplataforma, diagnóstico, documentación viva y proceso de releases.

La recomendación es no abrir más de dos frentes grandes al mismo tiempo. Cada versión debe combinar una mejora visible para el usuario con una reducción concreta de deuda técnica.

2. Principios de producto

2.1 Posicionamiento

CryptoCarver debe seguir siendo una herramienta de laboratorio y pruebas, no un producto que prometa custodia de claves de producción. Esa posición permite mostrar claves, datos intermedios y pasos del algoritmo, que son precisamente parte de su valor didáctico y diagnóstico.

2.2 Principios de diseño

- **Bytes primero:** toda operación debe distinguir claramente entre bytes, texto, codificación y representación.
- **Sin conversiones implícitas:** cuando exista una transformación adicional, como EBCDIC, compresión o canonicalización, debe aparecer como opción explícita.
- **Resultado explicable:** además del resultado, la aplicación debe mostrar algoritmo, parámetros, tamaños, formatos y advertencias relevantes.
- **Reproducibilidad:** una operación del histórico debe poder reconstruirse o exportarse como receta.
- **Interoperabilidad antes que cantidad:** una operación se considera completa cuando funciona con herramientas o vectores externos.
- **Modo laboratorio visible:** las decisiones pensadas para test, como guardar claves privadas en el histórico, deben estar claramente etiquetadas.
- **Modularidad:** cada modulo funcional debe poder evolucionar sin aumentar el tamaño y acoplamiento del controlador principal.

3. Estado actual resumido

Área	Situación actual	Riesgo o límite principal
Conversiones	Hex, Base64, binario, decimal, texto y EBCDIC multicodepage	Dos implementaciones de DataConverter y formatos globales compartidos por demasiadas pantallas
Cifrado	DES, 3DES, AES y varios modos	Faltan más pruebas negativas, operaciones con archivos grandes y AEAD moderno adicional
Claves	Generación, KCV, componentes, KDF y TR-31	TR-31 conserva parsing simplificado de bloques opcionales y falta un modelo único de clave
Firmas y certificados	RSA, ECDSA, EdDSA, certificados, CSR, CMS	Validación e interoperabilidad desigual según operación
JOSE	JWT, JWE, JWK y JWKS	Hay caminos EC simplificados o incompletos y falta cobertura de serializaciones avanzadas
Pagos y EMV	PIN blocks, CVV, PVV, ARQC/ARPC y derivaciones	Necesita más vectores por perfil y mejor separación entre variantes del estándar
XAdES	Firma B/T/LT/LTA, TSA configurable y truststore	Debe probarse con TSA y cadenas reales, y ofrecer informes más detallados
Post-cuántica	ML-KEM, ML-DSA, SLH-DSA, PEM e histórico	Conviven nombres pre-estándar y estándar; faltan KAT e interoperabilidad externa
UI	Rail lateral, panel de operaciones, inspector e histórico	ModernMainController concentra demasiada lógica y los formatos globales crean ambigüedad
Pruebas	Tests para hash, dígitos, EBCDIC, PQC y XAdES	Cobertura baja respecto al número de módulos y escasas pruebas UI/interoperabilidad

4. Arquitectura objetivo

4.1 Estructura modular

Cada función debe organizarse en cuatro piezas:

```
Vista JavaFX/FXML
-> Controlador del módulo
-> Servicio de operación
-> Modelo de entrada y resultado
```

El controlador principal debe limitarse a navegación, cabecera, inspector, histórico y ciclo de vida de los módulos. No debe contener implementaciones criptográficas ni cientos de handlers específicos.

4.2 Componentes comunes propuestos

- **OperationRegistry**: registro central de operaciones, categorías, iconos y navegación.
- **CodecRegistry**: conversión uniforme entre Hex, Base64, binario, decimal, texto y charsets.
- **OperationRequest** y **OperationResult**: modelos comunes para entradas, salidas, advertencias, detalles e intermedios.
- **HistoryCommand**: receta serializable capaz de reabrir y repetir una operación.
- **KeyMaterial**: modelo común para claves en memoria, PEM, JWK, PKCS#8, PKCS#12 y referencias PKCS#11.
- **ValidationReport**: resultado común para firmas, certificados, MAC y verificaciones.
- **AppSettings**: preferencias de TSA, truststores, idioma, tema, visibilidad de secretos y directorios recientes.
- **ErrorPresenter**: errores técnicos con causa, sugerencia y opción de copiar diagnóstico.

4.3 Limpieza técnica prioritaria

- Unificar los dos DataConverter existentes y eliminar el paquete duplicado.
- Dividir ModernMainController por módulos FXML independientes.

- Sacar clases de depuración de `src/main/java/com/cryptoforge/test`.
- Evitar versionar artefactos de `target/` y revisar `.gitignore`.
- Usar `--release 17` o migrar de forma planificada a Java 21 LTS.
- Añadir una implementación SLF4J para eliminar el logger NOP y controlar niveles.
- Centralizar versiones Maven y activar comprobación de dependencias vulnerables.
- Definir una política de excepciones: validación de entrada, error criptográfico, error de fichero y error de red.

5. Roadmap por fases

Fase 0 - Estabilización y base de calidad

Duración orientativa: 2 a 3 semanas

Objetivo: asegurar que el estado actual sea reproducible antes de ampliar alcance.

Funcionalidad

- Inventariar todas las operaciones visibles y marcar cada una como completa, experimental o pendiente.
- Corregir rutas que anuncian soporte no terminado, especialmente JOSE EC y TR-31 opcional.
- Unificar conversiones y formatos para eliminar resultados distintos entre pantallas.
- Revisar que toda operación actualice inspector, estado e histórico de forma consistente.
- Añadir un panel de diagnóstico con Java, JavaFX, proveedor BC, DSS, sistema y arquitectura.

Interfaz

- Etiqueta visible `Laboratory/Test Tool` en cabecera o About.
- Estado vacío y ayuda contextual en todos los módulos.
- Convención única para botones: ejecutar, verificar, importar, exportar y limpiar.
- Indicadores de tarea para operaciones que pueden bloquear la UI.
- Normalizar títulos, subtítulos, iconos, tamaños y scroll de cada módulo.

Calidad

- Test de carga de todos los FXML.
- Smoke test de cada handler principal.
- Build limpio en macOS, Windows y Linux.
- Eliminar advertencias evitables de compilación.

Criterio de salida

- Cero excepciones al navegar o ejecutar ejemplos básicos.
- Todas las operaciones visibles tienen estado documentado.
- `mvn clean test` y empaquetado multiplataforma pasan de forma repetible.

Fase 1 - Experiencia de usuario y modelo de operaciones

Duración orientativa: 3 a 5 semanas

Objetivo: convertir la interfaz en un laboratorio coherente y predecible.

Mejoras principales

- Sustituir formatos globales por formatos asociados a cada operación, manteniendo una opción de sincronización global.
- Añadir una barra de contexto que resuma `Entrada -> Transformación -> Salida`.

- Hacer que el inspector muestre bytes reales, no aproximaciones basadas en caracteres.
- Permitir copiar cada campo, no solo la salida completa.
- Añadir redimensionado persistente de paneles y modo de pantalla compacta.
- Búsqueda global por algoritmo, estándar, alias y caso de uso.
- Favoritos y operaciones recientes en la navegación.
- Atajos de teclado para ejecutar, limpiar, intercambiar entrada/salida y copiar.
- Tema claro, oscuro y alto contraste.
- Escalado de fuente y accesibilidad de teclado completa.
- Etiquetas y tooltips en español e inglés mediante recursos i18n.

Histórico y sesiones

- Histórico con columnas configurables, filtros por módulo, fecha y resultado.
- Vista detallada estructurada en vez de JSON plano.
- Repetición real de operaciones mediante HistoryCommand.
- Comparación entre dos ejecuciones.
- Exportación de una operación como Markdown, JSON o receta CLI.
- Perfil de visibilidad de secretos: completo, parcialmente oculto o redactado.
- Límite y rotación configurables del historial.

Criterio de salida

- Un usuario puede comprender entradas y salidas sin conocer la implementación interna.
- El historial reproduce al menos el 80 % de las operaciones soportadas.
- Navegación y uso completo con teclado.

Fase 2 - Conversiones, inspección de bytes y archivos

Duración orientativa: 3 a 4 semanas

Objetivo: hacer de CryptoCarver una herramienta fuerte de diagnóstico de datos criptográficos.

> Decisión de interfaz: las transformaciones permanecerán en **Manual Conversion**. El visor hexadecimal, comparador de charsets, entropía/frecuencia, visualización de controles, XOR y diff se consolidarán en una herramienta independiente de **Byte Inspector**, para no mezclar análisis con conversión.

Conversiones de datos

- Vista hexadecimal con offset, ASCII, EBCDIC, selección de rango y resaltado.
- Comparador simultáneo de interpretaciones UTF-8, Latin-1, Windows-1252 y codepages EBCDIC.
- Detección orientativa de charset con puntuación y aviso de que no es concluyente.
- Visualización de caracteres de control (NUL, CR, LF, TAB, separadores EBCDIC).
- Conversión endian de enteros de 16, 32, 64 y 128 bits.
- Enteros con y sin signo, BCD empaquetado/no empaquetado y COMP-3.
- TLV genérico, BER-TLV, EMV TLV y parsing de longitudes.
- Base32, Base58, Base58Check, Base64URL, URL encoding y Quoted-Printable.
- XOR visual entre dos buffers y diff byte a byte.
- Entropía, frecuencia de bytes y estimación básica de aleatoriedad.
- Compresión/descompresión gzip, zlib y raw DEFLATE antes o después de cifrar.

Archivos

- Arrastrar y soltar archivos en cualquier campo compatible.
- Vista previa sin cargar completamente archivos grandes.
- Hash, MAC, cifrado y conversión por streaming.
- Comparacion de archivos y localizacion del primer byte diferente.
- Exportacion con extensión sugerida y aviso de sobrescritura.

EBCDIC

- Mantener activacion explícita mediante checkbox.
- Añadir favoritos de codepage y búsqueda dentro del selector.
- Ofrecer vista previa paralela de los tres codepages más probables.
- Soportar BCD, zoned decimal y campos COBOL habituales.
- Preservar y mostrar espacios, padding y caracteres no imprimibles.

Criterio de salida

- Conversiones round-trip verificadas por charset y formato.
- Archivos de al menos 1 GB procesados sin cargarlos completamente en memoria.
- Resultados reproducibles desde histórico o receta.

Fase 3 - Criptografía simétrica, MAC y derivación

Duración orientativa: 4 a 6 semanas

Objetivo: ampliar algoritmos y, sobre todo, hacer visibles las condiciones de uso correcto.

Cifrado

- ChaCha20-Poly1305 y, si el proveedor lo permite, XChaCha20-Poly1305.
- AES Key Wrap y AES Key Wrap with Padding.
- Operaciones de archivo por streaming para CBC, CTR, GCM y ChaCha20.
- Generacion automática y exportación separada de IV, nonce, AAD y tag.
- Modo educativo que muestre bloques, padding y cadena CBC.
- Advertencias cuando se reutiliza nonce/IV dentro de una sesion.
- Validación estricta de tamaño de clave, IV y tag antes de ejecutar.

MAC

- CMAC AES/3DES, HMAC completo y variantes truncadas configurables.
- ISO 9797-1 con método de padding y longitud de salida explicitos.
- GMAC y Poly1305.
- Comparacion en tiempo constante para verificaciones.
- Vectores de prueba visibles y cargables.

KDF y contraseñas

- PBKDF2 con todas las variantes SHA relevantes.
- scrypt y Argon2id para laboratorio de password hashing.
- HKDF Extract/Expand por separado.
- NIST SP 800-108 Counter/Feedback/Double-Pipeline.

- ANSI X9.63 KDF y concatenation KDF.
- Diagrama de inputs: secreto, salt, info, contexto, contador y longitud.
- Comparador de parámetros y coste temporal.

Criterio de salida

- Vectores conocidos para cada algoritmo y modo.
- Pruebas negativas de IV, tag, padding y claves incorrectas.
- Ninguna operación insegura se presenta sin advertencia contextual.

Fase 4 - Claves, certificados y proveedores

Duración orientativa: 4 a 6 semanas

Objetivo: unificar el ciclo de vida de claves y mejorar el diagnóstico PKI.

Gestión de claves

- Importar/exportar PEM, DER, PKCS#8, PKCS#12, JKS, JCEKS y JWK.
- Claves privadas PEM cifradas y selección del algoritmo de protección.
- Identificación por huella, algoritmo, tamaño, curva y usos permitidos.
- Comparar clave pública con privada o certificado.
- Detectar claves débiles, paridad DES y componentes duplicados.
- Generar componentes con ceremonias simuladas y doble control.
- Borrado explícito de material sensible en memoria cuando sea viable.

PKCS#11 y HSM

- Primera fase: HSM simulado con claves no exportables.
- Segunda fase: proveedor PKCS#11 configurable para tokens reales de laboratorio.
- Enumerar slots, mecanismos, objetos y certificados.
- Firmar, verificar, cifrar y calcular MAC sin extraer la clave.
- Perfil de configuración por proveedor y diagnóstico de librería nativa.

Certificados

- Constructor de CSR con extensiones y SAN completos.
- Generación de CA raíz/intermedia y certificados finales de laboratorio.
- Constructor visual de cadenas y diagnóstico de cada salto.
- OCSP, CRL, AIA y descarga de emisores con cache.
- Comparador de certificados y diff de extensiones.
- Linter de fechas, KeyUsage, EKU, BasicConstraints, nombres y algoritmos.
- Truststores guardados como perfiles reutilizables.
- Exportación de informes PKI en Markdown/PDF.

Criterio de salida

- El mismo KeyMaterial funciona en cifrado, firmas, JOSE, CMS, XAdES y PQC.
- Validación de cadena con explicación clara del punto de fallo.
- Prueba de integración con al menos un token PKCS#11 software.

Fase 5 - Firmas avanzadas y servicios de confianza

Duración orientativa: 5 a 8 semanas

Objetivo: completar interoperabilidad de firmas y validación a largo plazo.

XAdES

- Probar Baseline B, T, LT y LTA con TSA pública y TSA corporativa.
- Boton Test TSA con latencia, certificado, algoritmo y política.
- Perfiles guardados de TSA con autenticación opcional.
- Packaging enveloped, enveloping y detached.
- Multiples firmas, countersignature y cofirma.
- Política de firma explícita y roles/commitment types.
- Trusted Lists europeas opcionales y cache controlada.
- Informes Simple, Detailed y ETSI Validation Report.
- Vista jerarquica de referencias, transforms, digest y SignedProperties.
- Validación externa cruzada con DSS demo o herramienta equivalente.

CMS/CAAdES/PAAdES/ASiC

- Completar CAAdES Baseline B/T/LT/LTA reutilizando DSS.
- PAAdES para PDF de laboratorio con firma visible opcional.
- ASiC-S y ASiC-E con XAdES/CAAdES.
- Firma detached de grandes archivos por streaming.
- Timestamp RFC 3161 independiente para cualquier hash o fichero.
- Verificador generico que detecte formato de firma automaticamente.

Criterio de salida

- Matriz automatizada de formato, nivel, packaging y TSA.
- Informes que separen integridad, confianza, revocacion, tiempo y cualificación.
- Interoperabilidad probada con al menos una herramienta externa por formato.

Fase 6 - Post-cuántica e hibridación

Duración orientativa: 4 a 6 semanas

Objetivo: alinear el modulo con los nombres y flujos estandarizados.

Alcance funcional

- Presentar ML-KEM, ML-DSA y SLH-DSA como nombres principales.
- Mantener Kyber, Dilithium y SPHINCS+ solo como alias de compatibilidad claramente etiquetados.
- Vectores KAT oficiales para cada conjunto de parámetros.
- Importación/exportación con detección automática del conjunto de parámetros.
- Benchmark de keygen, encapsulación, firma, verificación, tamaños y memoria.
- Flujo didáctico completo de KEM: emisor, receptor, encapsulación y clave derivada.
- Demostracion hibrida clásica + PQC, sin presentarla como protocolo normativo.
- Comparador de tamaños RSA/ECC/PQC.
- SLH-DSA completo y evaluación futura de FN-DSA cuando el ecosistema sea estable.

- Perfil de compatibilidad por proveedor Bouncy Castle.

Integraciones futuras

- Certificados experimentales con claves PQC cuando las librerías y perfiles sean interoperables.
- Firmas CMS/JOSE experimentales con identificadores estandarizados.
- Recetas de migración y crypto-agility.

Criterio de salida

- Round-trip y KAT para todos los parámetros publicados.
- No mezclar nombres pre-estándar y estándar sin explicación visible.
- Resultados comparables con una segunda implementación externa.

Fase 7 - Pagos, EMV y TR-31

Duración orientativa: 6 a 10 semanas

Objetivo: convertir el área de pagos en un laboratorio de referencia con perfiles reproducibles.

TR-31

- Completar bloques opcionales y validación de headers.
- Matriz de versiones A/B/C/D, algoritmo, modo de uso y exportabilidad.
- Construcción visual del header y explicación campo a campo.
- Vectores positivos y negativos por versión.
- Importación/exportación por lotes.
- Comparación con una implementación de referencia cuando la licencia lo permita.

DUKPT y derivaciones

- DUKPT TDES completo: IPEK, contador, session key y variantes PIN/MAC/DATA.
- AES DUKPT para perfiles de laboratorio.
- Árbol visual de derivación y seguimiento del KSN.
- Validación de contador y detección de reutilización.

EMV

- Derivación Option A y B con perfiles claros.
- ARQC/ARPC por esquema y método con campos etiquetados.
- Construcción y parsing de CDOL1/CDOL2.
- EMV TLV integrado con diccionario de tags.
- Script MAC y secure messaging.
- CVN/perfiles configurables y recetas guardadas.

PIN y tarjetas

- Matriz ISO 9564 con diagramas de cada PIN block.
- Traducción de PIN blocks con claves y PAN diferentes.
- IBM 3624, PVV, CVV/iCVV/dCVV con pasos intermedios opcionales.
- Pistas 1/2, LRC y validaciones de formato.
- Datos ficticios generables para escenarios de prueba.

Criterio de salida

- Cada operación tiene al menos dos vectores positivos y dos negativos.
- Perfiles y supuestos aparecen en pantalla y en el histórico.
- Ningun resultado depende de heurísticas silenciosas.

Fase 8 - JOSE, ASN.1 y protocolos

Duración orientativa: 4 a 6 semanas

Objetivo: cerrar operaciones incompletas y mejorar inspección estructurada.

JOSE

- Completar firma EC y conversión EC PEM/JWK para curvas soportadas.
- JWS Compact y JSON Serialization, incluyendo múltiples firmas.
- Payload detached y b64=false.
- JWE con RSA-OAEP, AES-KW, ECDH-ES y PBES2.
- Nested JWT configurable: firmar luego cifrar y proceso inverso.
- Validación temporal con reloj configurable y tolerancia de clock skew.
- Constructor de claims y perfiles OAuth/OIDC de laboratorio.
- JWKS con rotación, kid, thumbprint y selección determinista.
- Diagnostico de algoritmo confuso o incompatible con la clave.

ASN.1

- Validador DER frente a BER permisivo.
- Editor árbol y re-encode con diff hexadecimal.
- Esquemas PKCS#1, PKCS#8, PKCS#10, PKCS#7, X.509, CRL, OCSP y TSTInfo.
- Registro OID ampliable por fichero.
- Búsqueda por offset desde hex hacia nodo ASN.1.
- Exportación del árbol a JSON y Markdown.

Criterio de salida

- Cobertura completa de los algoritmos anunciados en UI.
- Tests cruzados con bibliotecas JOSE externas.
- Parser ASN.1 resistente a entradas truncadas o malformadas.

Fase 9 - Automatización, lotes y API local

Duración orientativa: 4 a 6 semanas

Objetivo: reutilizar el motor sin depender siempre de la interfaz gráfica.

Recetas

- Formato JSON versionado para describir una operación.
- Exportar cualquier ejecución como receta.
- Importar, validar, previsualizar y ejecutar recetas.
- Variables, referencias a ficheros y valores generados.
- Cadena de operaciones: decode -> decrypt -> EBCDIC -> parse TLV.

Batch

- Entradas CSV/JSONL y mapeo de columnas a parámetros.
- Progreso, cancelación, errores por fila y resumen final.
- Salida CSV/JSONL con resultados y diagnosticos.
- Paralelismo configurable para operaciones seguras.

CLI y API

- CLI nativa sobre los mismos servicios que la GUI.
- Salida humana y JSON estable.
- Codigos de salida documentados.
- Servidor REST local opcional, desactivado por defecto.
- OpenAPI generado y límite explícito de tamaño/peticiones.

Criterio de salida

- GUI y CLI producen resultados byte a byte identicos.
- Una receta puede compartirse sin incluir secretos si se usa modo redactado.
- Los lotes son cancelables y no bloquean la UI.

Fase 10 - Distribución, documentación y ecosistema

Duración orientativa: 3 a 5 semanas iniciales y mantenimiento continuo

Objetivo: facilitar instalación, soporte y contribución.

Distribución

- Instaladores nativos con jpackage para macOS, Windows y Linux.
- Runtime Java incluido para evitar diferencias de entorno.
- Firma/notarizacion opcional de binarios cuando exista infraestructura.
- Checksums SHA-256 y SBOM por release.
- Versión visible en UI y diagnóstico exportable.
- Canal estable y canal experimental.

Documentación

- Guía de inicio rapido por caso de uso.
- Manual por modulo con ejemplos reproducibles.
- Catalogo de formatos, charsets y alias.
- Documento de diferencias entre herramienta de test y uso productivo.
- Changelog por versión con migraciones y breaking changes.
- Capturas actualizadas automaticamente cuando sea viable.

Extensibilidad

- SPI de operaciones para añadir módulos sin modificar el controlador principal.
- Paquetes de vectores de prueba instalables.
- Perfiles de empresa locales: TSA, truststore, codepages y algoritmos permitidos.
- Evaluar plugins solo después de estabilizar OperationRegistry y modelos comunes.

6. Backlog transversal priorizado

Prioridad	Iniciativa	Valor	Esfuerzo	Dependencias
P0	Unificar DataConverter y codecs	Evita inconsistencias y errores runtime	M	Ninguna
P0	Dividir ModernMainController	Reduce riesgo de cada cambio UI	L	OperationRegistry básico
P0	Test de carga FXML y smoke UI	Detecta fallos antes de ejecutar manualmente	M	JavaFX test harness
P0	Matriz de estado de operaciones	Evita anunciar soporte incompleto	S	Inventario
P0	Vectores conocidos por modulo	Aumenta confianza en resultados	L	Catalogo de tests
P1	Formatos por operación	Elimina ambigüedad de entrada/salida	M	CodecRegistry
P1	Histórico reproducible	Convierte sesiones en casos de prueba	L	OperationRequest/Result
P1	XAdES interoperable B/T/LT/LTA	Alto valor profesional	L	Perfiles TSA/truststore
P1	TR-31 completo	Alto valor en pagos	L	Vectores y parsing
P1	PQC KAT y nomenclatura estándar	Alineación técnica y educativa	M	Proveedor BC
P1	Conversor/inspector de bytes avanzado	Uso frecuente y transversal	M	CodecRegistry
P1	Empaquetado con runtime incluido	Reduce incidencias de instalación	M	Build estable
P2	PKCS#11/HSM software	Amplia escenarios profesionales	L	KeyMaterial
P2	DUKPT TDES/AES	Completa pagos	L	Modulo pagos estable
P2	PADES/CADES/ASiC	Extiende firmas europeas	XL	DSS estable
P2	CLI y recetas	Automatización y reproducibilidad	L	Servicios desacoplados
P2	Batch CSV/JSONL	Productividad en pruebas	M	CLI/OperationRequest
P2	Temas, i18n y accesibilidad	Mejora adopción	M	UI modular
P3	API REST local	Integración externa	M	CLI/servicios comunes
P3	Sistema de plugins	Extensibilidad	XL	API interna estable
P3	Protocolos híbridos PQC	Laboratorio avanzado	L	PQC maduro

Escala de esfuerzo: S = pocos días, M = 1-2 semanas, L = 3-6 semanas, XL = más de 6 semanas o investigación significativa.

7. Estrategia de pruebas

7.1 Pirámide de pruebas

- **Unitarias:** codecs, validadores, algoritmos y modelos.
- **Property-based:** round-trip de conversiones, claves, padding y serializaciones.
- **Vectores conocidos:** NIST, RFC, EMV y otros vectores legalmente utilizables.
- **Negativas:** datos truncados, formatos invalidos, claves incorrectas, tags y firmas alteradas.
- **Integración:** keystores, truststores, TSA, OCSP, CRL, ficheros y proveedores.
- **Interoperabilidad:** OpenSSL, herramientas DSS, bibliotecas JOSE y segunda implementación PQC.
- **UI:** carga FXML, navegación, visibilidad de contenedores, handlers e histórico.
- **Regresión visual:** capturas de pantallas principales en tamaños representativos.
- **Rendimiento:** archivos grandes, PQC, KDF costosas y lotes.
- **Fuzzing:** parsers ASN.1, TLV, JWT, certificados, TR-31 y entradas hex/Base64.

7.2 Matriz mínima por operación

Cada operación nueva debe incluir:

- Un caso positivo mínimo.
- Un vector conocido independiente cuando exista.
- Un round-trip cuando la operación sea reversible.
- Dos entradas invalidas significativas.
- Comprobacion de inspector e histórico.
- Documentación del formato exacto de entrada y salida.
- Prueba con build limpio.

7.3 Calidad de release

- Todos los tests verdes.
- Sin excepciones no capturadas durante smoke test.
- Dependencias revisadas.
- Documentación y capturas actualizadas.
- Instaladores probados en las tres plataformas.
- Changelog y notas de compatibilidad.

8. Modelo de versiones recomendado

Versión	Tema principal	Resultado esperado
2.2.x	Estabilización de PQC, XAdES, EBCDIC y UI	Estado actual fiable y documentado
2.3	Arquitectura UI y codecs	Modulos desacoplados, formatos claros e histórico consistente
2.4	Inspector de bytes y conversiones	Herramienta avanzada de diagnóstico de datos
2.5	Firmas y confianza	XAdES maduro, TSA/truststore e informes completos
2.6	Pagos y TR-31	Vectores, optional blocks y perfiles EMV/PIN
2.7	PQC estandarizado	KAT, benchmarks y compatibilidad por proveedor
2.8	Automatización	Recetas, CLI y batch
3.0	Plataforma modular	Instaladores, API interna estable y extensibilidad

Las versiones son orientativas. Una versión no debe cerrarse por fecha si no cumple sus criterios de salida.

9. Siguiente sprint recomendado

Duración sugerida: 2 semanas.

1. Crear inventario de operaciones con estado estable, experimental o incompleto.
2. Unificar `com.cryptoforge.util.DataConverter` y `com.cryptoforge.utils.DataConverter`.
3. Añadir test que cargue `main-view-modern.fxml` y valide todos los handlers.
4. Extraer Post-Quantum y XML Security a FXML/controladores completamente independientes.
5. Crear `CodecRegistry` y migrar Manual Conversión como primer consumidor.
6. Corregir los bytes del inspector para que procedan del resultado real.
7. Convertir el detalle del histórico de JSON plano a una vista clave/valor con campos multilinea.
8. Añadir ejemplos integrados: UTF-8, EBCDIC España, ML-KEM y XAdES-B.
9. Configurar SLF4J y un dialogo Copy `diagnostics`.
10. Actualizar README para reflejar el soporte realmente disponible.

Entregable del sprint

Una versión 2.2.x que arranque limpiamente, tenga conversiones coherentes, cargue todos los FXML bajo test y permita diagnosticar errores sin revisar la terminal.

10. Decisiones que conviene posponer

- No adoptar DSS release candidate como dependencia por defecto; evaluar primero la última versión estable compatible.
- No añadir plugins antes de estabilizar modelos de operación y navegación.
- No mezclar PQC con XAdES/CMS como funcionalidad principal hasta que identificadores y herramientas sean interoperables.
- No exponer una API REST antes de separar por completo servicios y UI.
- No prometer cumplimiento regulatorio; ofrecer evidencias, informes y referencias técnicas.
- No añadir más algoritmos legacy salvo que exista un caso de prueba concreto.

11. Indicadores de evolución

Indicador	Objetivo inicial
Operaciones con vector conocido	80 % en 6 meses
Operaciones reproducibles desde histórico	80 % en 4 meses
Modulos con controlador/FXML independiente	100 % en 6 meses
Errores UI no capturados en smoke test	0
Tiempo de arranque en equipo de referencia	Menos de 5 segundos tras build
Archivos procesables por streaming	Al menos 1 GB
Releases con instaladores en 3 plataformas	100 % desde 2.5
Documentación de entradas/salidas	100 % de operaciones visibles

12. Definition of Done

Una mejora se considera terminada cuando:

- La operación tiene entradas, formatos y unidades visibles.
- Valida errores antes de invocar el motor criptográfico.
- No bloquea la interfaz durante tareas largas.
- Muestra resultado, detalles, advertencias y pasos relevantes.
- Registra una entrada útil y reproducible en el histórico.
- Incluye tests positivos, negativos y vector externo cuando exista.
- Tiene documentación y ejemplo cargable.
- Funciona tras `mvn clean test` y en el paquete de escritorio.
- No introduce secretos en logs involuntariamente; la visibilidad de laboratorio es explícita.
- Ha sido probada visualmente en resolución normal y compacta.

13. Referencias técnicas

- NIST Post-Quantum Cryptography Project: <https://csrc.nist.gov/Projects/post-quantum-cryptography>

- NIST FIPS 203, 204 y 205:
<https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>
- European Commission Digital Signature Service:
<https://ec.europa.eu/digital-building-blocks/DSS/webapp-demo/doc/dss-documentation.html>
- European Commission eSignature DSS overview:
<https://ec.europa.eu/digital-building-blocks/sites/spaces/DIGITAL/pages/467109107/Digital+Signature+Service+-+DSS>
- OWASP Cryptographic Storage Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html
- OWASP Key Management Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Key_Management_Cheat_Sheet.html

14. Mantenimiento del roadmap

Este documento debe revisarse al inicio de cada versión y al cerrar cada sprint. Las nuevas ideas se añaden primero al backlog, con valor, esfuerzo, riesgo y dependencia. Solo pasan a una fase cuando tienen criterio de aceptación y un responsable de validación.

La regla principal del roadmap es sencilla: **cada nueva capacidad debe hacer CryptoCarver más fiable, más explicable o más reutilizable; idealmente las tres cosas.**